🚀 Lecture 02.10: Operator Precedence & Expression Trees

📌 Executive Architectural Summary

When an expression contains multiple operators, operator precedence defines the syntactic binding power of each token, determining the hierarchical shape of the Abstract Syntax Tree (AST).

🔬 Core Engineering Mechanics

1. The Complete Precedence Hierarchy (Highest to Lowest)

1. **Parentheses:** `()` (Overrides all precedence)
2. **Exponentiation:** `**` (Right-to-left associative: $2^{3^2} = 2^9 = 512$)
3. **Unary Operators:** `+x`, `-x`, `~x`
4. **Multiplication, Division, Floor, Modulo:** `*`, `/`, `//`, `%` (Left-to-right)
5. **Addition, Subtraction:** `+`, `-` (Left-to-right)
6. **Bitwise Shifts:** `<<`, `>>`
7. **Bitwise AND / XOR / OR:** `&`, `^`, `|`
8. **Relational / Comparison:** `<`, `<=`, `>`, `>=`, `!=`, `==`, `is`, `in`
9. **Logical NOT:** `not`
10. **Logical AND:** `and`
11. **Logical OR:** `or`
12. **Assignments:** `=`, `+=`, `-=`, etc.

💻 Production Python Implementation

```
# Expression Trace: x = 10 + 3 * 2 ** 2
# Step 1: Exponentiation: 2 ** 2 = 4
# Step 2: Multiplication: 3 * 4 = 12
# Step 3: Addition: 10 + 12 = 22
x = 10 + 3 * 2 ** 2
print("10 + 3 * 2 ** 2 =", x)

# Exponentiation Right-to-Left Associativity:
# 2 ** 3 ** 2 = 2 ** (3 ** 2) = 2 ** 9 = 512
```

```
p = 2 ** 3 ** 2
print("2 ** 3 ** 2 =", p)

# Parentheses override
y = (10 + 3) * (2 ** 2) # 13 * 4 = 52
print("(10 + 3) * (2 ** 2) =", y)
```